Process & Decision Documentation

Side Quest Week 9 — Fox Rescue: Debug Screen + Second Level

Project/Assignment Decisions

I decided to build the debug screen as a full-screen overlay (toggled with T) rather than a small corner HUD, because the Week 9 prompt emphasizes features that help with testing and balancing a game. A full-screen panel provides enough space to display toggleable options, runtime stats, and playtest data side by side, making it a practical tool for game development rather than just a visual indicator. I also chose moon gravity (30% of normal) as the primary toggle because it directly changes how the player navigates levels, which is the most useful modifier for testing platformer layouts and reaching hard-to-access areas.

For the bonus, I added a second level ("The Deep Woods") with a different tilemap layout and a darker background color, loaded via pressing N on the win screen. This required building a full level transition system that tears down and rebuilds all sprites, groups, and event subscriptions for the new level while preserving the original canvas resolution, integer scaling, and control feel from Level 1.

Role-Based Process Evidence

Name: Jimin Kim, j243kim, 21062367 **Role(s):** Individual (sole developer) **Primary responsibility for this work:** All code, design, and documentation

Goal of Work Session

Adding a debug screen with toggleable gameplay modifiers (moon gravity, invincibility, slow motion) and cumulative playtest stat tracking to the existing Week 6 platformer. Bonus: designing and implementing a second level with a level transition system.

Tools, Resources, or Inputs Used

- Claude Code (Claude Opus 4.6) — used for implementation assistance
- GPT Model / Codex — used for follow-up debugging of the remaining level 2 control issue and for documentation updates
- Prior Week 6 codebase (own code from Side Quest 6)
- p5.js and p5play documentation
- Lecture notes on debug and testing features

GenAI Documentation

Date Used: April 1-2, 2026

Tool Disclosure: Claude Code (Claude Opus 4.6, via Anthropic CLI) for implementation assistance and a second debugging session, and GPT Model / Codex for an intermediate debugging follow-up and documentation updates.

Purpose of Use: Used Claude Code to assist with implementing the debug screen features, wiring the toggle flags through the existing modular architecture (InputManager -> Game -> Level -> PlayerController), designing the second level's tilemap, and building the async level transition system in main.js. Later used GPT Model / Codex to investigate the remaining level 2 jump/motion bug, compare level 1 and level 2 behavior, test a main.js-only transition-path fix, and update the README and Process document. In a second Claude Code session, further investigated the level 2 control bug by simplifying `transitionToLevel()` to reuse `initRuntime()`, then diagnosed the root cause as stale p5play Group objects whose `.tile` registrations conflict with newly created groups during level transitions.

Summary of Interaction: I first provided Claude Code with the Side Quest 9 prompt and existing Week 6 codebase. The tool read through all source files to understand the architecture, then implemented: (1) an expanded DebugOverlay.js with toggle flags and stat tracking, (2) new input keys in InputManager.js, (3) debug flag wiring through Game.js and Level.js, (4) invincibility logic in PlayerController.js, (5) a second level tilemap in levels.json, (6) a `transitionToLevel()` function in main.js, and (7) "Press N for next level" UI in WinScreen.js. During follow-up debugging, GPT Model / Codex inspected `PlayerEntity.js`, `Level.js`, `Game.js`, `main.js`, and `levels.json`, first suspected grounded-detection issues in the player sensor, then tested a main.js-only change that rebuilt level transitions through the same runtime path as level 1. After I adjusted the level 2 start area to match level 1 and the bug still remained, the remaining likely cause was narrowed to the player's ground-sensor / `isGrounded()` logic in `PlayerEntity.js` rather than the level 2 tilemap layout itself. In a second Claude Code session, I suggested restructuring `transitionToLevel()` to reuse `initRuntime()` directly. Claude Code agreed and implemented the refactor, but the jump still did not work. Claude Code then read `TileBuilder.js` and traced how `new Tiles()` matches tilemap characters to groups, identifying the root cause: old p5play Group objects from level 1 persist in p5play's internal registry with their `.tile` property still set, so `Tiles()` spawns level 2 tiles into the stale empty groups instead of the newly created ones, leaving the new Level's solid groups empty and making `isGrounded()` always return false.

Human Decision Point(s): I chose which three debug features to implement (moon gravity, invincibility, slow motion) based on what would be most useful for actual game testing. I also reviewed the level 2 tilemap to verify it had enough leaves (17) relative to the win score (12) and that the player start position was above solid ground. During later debugging, I asked that level 1 behavior remain unchanged if possible, so the first retry focused on a main.js-only transition-path fix before considering edits to shared player code. I also compared level 1 and level 2 screenshots and adjusted the level 2 start area to match level 1 as a test case. In the second Claude Code session, I proposed the `initRuntime()` reuse approach based on my understanding that level 1's boot path works correctly, and Claude Code confirmed and implemented it before moving to deeper investigation of the stale groups issue.

Integrity & Verification Note: I verified the generated code by: reading through each modified file to check for logical errors, running a Python script to count tilemap entities and confirm both levels are completable, checking that the invincibility logic correctly prevents the death latch while still allowing damage events for visual feedback, and reviewing the level transition path against the original boot path to make sure the same runtime canvas/physics setup is applied in both cases. Additional review also confirmed that level bounds must be calculated from the widest tilemap row, not just the first one, so level 2 keeps the correct camera/layout footprint. Follow-up verification included syntax checks on `main.js` and `src/entities/PlayerEntity.js`, comparing level 1 and level 2 screenshots side by side, matching the level 2 start geometry to level 1, and confirming that level 2 still failed under those conditions. That ruled out the start-area tilemap as the sole cause. In the second Claude Code session, the `initRuntime()` reuse

refactor was verified to produce identical setup logic for both levels, and when the bug persisted, Claude Code traced through `TileBuilder.js` and the `new Tiles()` call to identify stale p5play Group registrations as the root cause — explaining why tiles were visible but the Level's solid groups were empty.

Scope of GenAI Use: GenAI assisted with the implementation of all code changes. The design decisions (which features, tilemap layout, gravity multiplier value) and verification were done by me. The original Week 6 codebase that this builds upon was also developed with GenAI assistance in a prior session. The later debugging strategy, comparison of likely causes, and documentation wording were also assisted by GPT Model / Codex but reviewed and adjusted by me. The second Claude Code session was used to implement the `initRuntime()` reuse refactor and to diagnose the stale p5play groups root cause through codebase exploration.

Limitations or Misfires: The initial implementation had several follow-up bugs or misfires: (1) `allSprites.removeAll()` was used for sprite cleanup during level transitions, but this method may not exist in all p5play v3 builds — replaced with a safe iteration loop. (2) The level transition code incorrectly overwrote the level index after `build()` had already set it correctly from the level ID, which would have broken the "next level" detection on level 2. (3) Level 2 also had inconsistent tilemap row widths while the loader sized the world from only the first row, which made the camera/world bounds slightly too narrow for the actual map data — fixed by normalizing tilemap rows before calculating bounds. (4) The remaining level 2 grounded-control bug was harder to diagnose than expected. An earlier explanation treated it as only a transition/runtime setup issue, but later debugging showed that reusing `initRuntime()` during transitions and matching the level 2 starting geometry to level 1 still did not solve the problem. (5) A second Claude Code session identified the actual root cause: stale p5play Group objects. When `transitionToLevel()` removes all sprites, the Group objects themselves persist in p5play's internal registry with their `.tile` property still set. When `new Tiles()` runs for level 2, it matches tile characters against the old empty groups instead of the new ones, leaving the new Level's solid groups empty and causing `isGrounded()` to always return false. The fix is to call `group.remove()` on each old Level group during the transition cleanup phase before building the new level.

Summary of Process (Human + Tool)

1. Started by reading the Week 9 prompt requirements: debug screen with moon gravity toggle + at least one other feature, bonus for adding a level.
2. Read through all existing source files to understand the modular architecture (main.js → Game.js → Level.js → PlayerController/entities).
3. Identified the key integration points: InputManager for new keys, DebugOverlay for the UI, Game.js for flag wiring, Level.js for physics/invincibility, PlayerController for damage prevention.
4. Implemented the debug screen with three toggles, runtime stats, playtest stat tracking, and event log.
5. Built the second level tilemap, verified entity counts.
6. Built the async level transition system that cleanly tears down and rebuilds the world.
7. Revisited the remaining level 2 control bug in a separate GPT Model / Codex debugging session and first tested a main.js-only fix that forced level transitions to reuse the same runtime creation path as level 1.
8. Compared level 1 and level 2 screenshots/tilemaps, matched the level 2 start area to level 1, and narrowed the likely remaining cause to grounded detection in `PlayerEntity.js` rather than the level layout itself.

9. In a second Claude Code session, simplified `transitionToLevel()` to delegate to `initRuntime()` directly, eliminating ~60 lines of duplicated setup logic. When the bug persisted, Claude Code read `TileBuilder.js` and traced the `new Tiles()` tile-character matching.
10. Identified the root cause as stale p5play Group objects whose `.tile` registrations survive sprite removal. The fix is to call `group.remove()` on each old Level group during the transition cleanup phase before `initRuntime()` builds new groups.

Decision Points & Trade-offs

1. **Debug toggles work globally (not just when debug screen is visible):** I considered making the toggles only work when the debug screen is open, but decided they should persist even when the screen is closed. This is more useful — you can toggle moon gravity, close the debug screen, and play normally to test how the floaty physics feels without the overlay blocking the view.

2. **Invincibility auto-heals rather than preventing damage entirely:** Rather than suppressing the damage callback, invincibility resets health each frame and blocks the death latch. This means the player still sees damage feedback (screen shake, particles, sound) but cannot die. This is intentional — during playtesting, it is useful to see *when* you would take damage even if you cannot die.

Verification & Judgement

- Verified both levels are completable by counting leaves vs. win scores (Level 1: 15/15, Level 2: 17/12).
- Confirmed player start positions are above solid ground tiles in both levels.
- Compared level 1 and level 2 start areas and confirmed that matching the level 2 starting geometry to level 1 did not restore jump or run animation.
- Verified the `initRuntime()` reuse refactor produces identical setup logic for both levels, ruling out the transition code path as the cause.
- Traced the `new Tiles()` tile-character matching in `TileBuilder.js` and confirmed that stale Group objects with `.tile` properties persist after sprite removal, causing tiles to spawn into wrong groups.
- Checked that debug flags propagate correctly through the architecture: InputManager → Game → Level → PlayerController.
- Reviewed the level transition code for resource cleanup (sprite removal, event re-subscription).
- Verified the assignment rubric requirements on implementation scope: the debug screen satisfies the core requirement, and the second level satisfies the bonus on content/design, though the remaining level 2 control bug still needs a final gameplay fix.

Limitations, Dead Ends, or Open Questions

- The level 2 control bug was diagnosed in a second Claude Code session: stale p5play Group objects from level 1 persist with their `.tile` property after sprite removal, causing `new Tiles()` to spawn level 2 tiles into the old empty groups. The fix is to call `group.remove()` on each old Level group during transition cleanup. The `transitionToLevel()` function has already been simplified to reuse `initRuntime()`; the remaining step is adding the group cleanup before `initRuntime()` runs.
- The level transition re-loads all image assets from URLs (they come from browser cache, but it is not instant). A loading indicator could improve the experience.
- Slow motion is implemented by skipping every other physics step (`frameCount % 2`), which means animations also appear choppy. A more refined approach would use a time scale multiplier, but this

was adequate for the debug feature scope.
- The second level reuses all the same assets (sprites, tiles, sounds) as level 1. A more polished version could introduce new enemy types or tile sets.

Appendix — Full GenAI Transcript

The following is the full transcript of the conversation with Claude Code (Claude Opus 4.6) used to implement the Week 9 Side Quest.

Prompt 1 (User)

> Read this current code files (side quest 06) and pdf as well, please modify the code to satisfy week 9 (March 19) sidequest as well as the bonus mark.

Attached: Side Quests.pdf (assignment spec, 7 pages)

Response 1 (Claude Code)

Claude Code read the entire codebase (main.js, index.html, levels.json, tuning.json, Game.js, DebugOverlay.js, Level.js, PlayerController.js, InputManager.js, WinScreen.js, LoseScreen.js, LevelLoader.js, BoarSystem.js, HUDRenderer.js, CameraController.js, AssetLoader.js, TileBuilder.js) to understand the architecture.

Then implemented all changes across 10 files:

1. DebugOverlay.js — Rewritten into a full debug screen with:

- Moon gravity toggle (G key, reduces gravity to 30%)
- Invincibility toggle (I key, auto-heals and prevents death)
- Slow motion toggle (M key, physics at half rate)
- Runtime stats: FPS, player position, score, health, gravity, entity counts, level name
- Playtest stats: deaths, damage taken, leaves collected, boars killed, jumps, session time
- Event log feed from EventBus
- Console logging of stats every 300 frames

Full rewritten file:

```
// src/DebugOverlay.js
// Debug screen (VIEW + SYSTEM tool).
//
// Week 9 Side Quest: full debug screen with toggleable features.
//
// Features:
// - Toggle moon gravity on/off (G key)
// - Toggle invincibility on/off (I key)
// - Toggle slow-motion on/off (M key)
// - Display real-time stats: FPS, player position, score, health, entity
counts
// - Track cumulative playtest stats: deaths, leaves collected, damage
```

```
taken
// - Event log feed from EventBus
//
// Responsibilities:
// - Render debug info in screen-space (camera.off())
// - Manage debug flags that Game/Level read each frame
// - Log events from EventBus (including wildcard "*")
//
// Non-goals:
// - Does NOT directly modify physics or entity state (Game/Level apply
the flags)

export class DebugOverlay {
  constructor() {
    this.enabled = false;
    this.moonGravity = false;
    this.invincible = false;
    this.slowMotion = false;
    this.stats = {
      deaths: 0,
      damagesTaken: 0,
      leavesCollected: 0,
      boarsKilled: 0,
      jumps: 0,
      sessionStartMs: Date.now(),
    };
    this.lines = [];
    this.maxLines = 6;
  }

  toggle() { this.enabled = !this.enabled; }
  toggleMoonGravity() { this.moonGravity = !this.moonGravity; }
  toggleInvincible() { this.invincible = !this.invincible; }
  toggleSlowMotion() { this.slowMotion = !this.slowMotion; }

  log(evt) {
    if (!evt) return;
    const name = evt.name ?? "";
    if (name === "player:died") this.stats.deaths++;
    if (name === "player:damaged") this.stats.damagesTaken++;
    if (name === "leaf:collected") this.stats.leavesCollected++;
    if (name === "boar:died") this.stats.boarsKilled++;
    if (name === "player:jumped") this.stats.jumps++;
    const msg = `${name}`;
    this.lines.unshift(msg);
    if (this.lines.length > this.maxLines) this.lines.length =
this.maxLines;
  }

  draw({ game } = {}) {
    if (!this.enabled) return;
    camera.off();
    // ... draws full debug panel with toggles, runtime stats, playtest
stats, event log
```

```
    camera.on();
  }

  _drawToggleLine(x, y, key, label, isOn) {
    fill(180);
    text(`[${key}]`, x, y);
    fill(isOn ? 0x00 : 255, isOn ? 255 : 255, isOn ? 0x7a : 255);
    text(`${label}: ${isOn ? "ON" : "OFF"}`, x + 22, y);
  }
}
```

2. InputManager.js — Added debug keys:

```
// Added to _prevDown:
moonGravity: false,
invincible: false,
slowMotion: false,
nextLevel: false,

// Added to _input:
moonGravityPressed: false,
invinciblePressed: false,
slowMotionPressed: false,
nextLevelPressed: false,

// Added key reads:
const moonGravityDown = kb.pressing("g");
const invincibleDown = kb.pressing("i");
const slowMotionDown = kb.pressing("m");
const nextLevelDown = kb.pressing("n");

// Added edge detection:
this._input.moonGravityPressed = moonGravityDown &&
!this._prevDown.moonGravity;
this._input.invinciblePressed = invincibleDown &&
!this._prevDown.invincible;
this._input.slowMotionPressed = slowMotionDown &&
!this._prevDown.slowMotion;
this._input.nextLevelPressed = nextLevelDown && !this._prevDown.nextLevel;
```

3. Game.js — Debug toggle wiring + level advancement:

Added to constructor:

```
this._levelIds = [];
this._currentLevelIndex = 0;
this._onLevelAdvance = null;
```

Added to `build()`:

```
const doc = this.pkg?.doc;
if (doc?.levels && Array.isArray(doc.levels)) {
  this._levelIds = doc.levels.map((l) => l.id);
  const curId = this.pkg?.level?.id ?? "";
  this._currentLevelIndex = Math.max(0, this._levelIds.indexOf(curId));
}
```

Added to `update()`:

```
// Debug toggle keys
if (this.debug) {
  if (inputSnap?.moonGravityPressed) {
    this.debug.toggleMoonGravity();
    this._applyGravity();
  }
  if (inputSnap?.invinciblePressed) this.debug.toggleInvincible();
  if (inputSnap?.slowMotionPressed) this.debug.toggleSlowMotion();
}

// Pass debug flags to Level
const debugFlags = this.debug
  ? { invincible: this.debug.invincible, slowMotion: this.debug.slowMotion
}
  : {};
this.level.update({ input: inputSnap, debugFlags });

// Next level (N key)
if (inputSnap?.nextLevelPressed && this.won && this._hasNextLevel()) {
  this.advanceLevel();
  return;
}
```

Added methods:

```
_applyGravity() {
  const baseGravity = Number(this.pkg?.world?.gravity ?? 10);
  const useMoon = this.debug?.moonGravity === true;
  world.gravity.y = useMoon ? baseGravity * 0.3 : baseGravity;
}

_hasNextLevel() {
  return this._currentLevelIndex < this._levelIds.length - 1;
}

getNextLevelId() {
  if (!this._hasNextLevel()) return null;
```

```
    return this._levelIds[this._currentLevelIndex + 1];
  }

  advanceLevel() {
    const nextId = this.getNextLevelId();
    if (!nextId) return;
    this._currentLevelIndex++;
    if (typeof this._onLevelAdvance === "function") {
      this._onLevelAdvance(nextId);
    }
  }
```

4. Level.js — Slow motion + invincibility passthrough:

```
// Changed signature:
update({ input, debugFlags = {} }) {

// Changed player update call:
this.playerCtrl.update({
  input, solids: this._solids(), bounds: this.bounds,
  won: this.won,
  invincible: debugFlags.invincible === true,
});

// Changed physics step:
if (debugFlags.slowMotion) {
  if (frameCount % 2 === 0) world.step();
} else {
  world.step();
}
```

5. PlayerController.js — Invincibility logic:

```
update({ input, solids, bounds, won, invincible = false }) {
  const p = this.player;
  if (!p?.sprite) return;

  // Invincibility: auto-heal and prevent death
  if (invincible && p.health < p.maxHealth) {
    p.health = p.maxHealth;
    p.pendingDeath = false;
    p.invulnTimer = 0;
    p.knockTimer = 0;
  }

  // ... existing logic ...

  // Death latch blocked when invincible
  if (!invincible && !p.dead && p.pendingDeath && grounded) {
```

```
    p.dead = true;
    // ...
  }
}
```

6. levels.json — Added second level:

```
{
  "id": "ex5_level2",
  "name": "The Deep Woods",
  "tiles": { "tileW": 24, "tileH": 24, "frameW": 32, "frameH": 32 },
  "tilemap": [
    "                        g  b  g  x         ",
    "            b  x      LgggggggR           ",
    "      x    LggR                     x      ",
    "     LR           f   x   b      x LR      ",
    "  f      b   x   LR  LgggR     LggR        ",
    "  LggR  LgR LgR           x  b      f      ",
    "               x   b  f  LgggggR   LgR     ",
    "     f x    b LgR  LggR              x     ",
    " x  LggR   LgR              b  x  f        ",
    "                x  f   LR  LgggggR LgR     ",
    "LgR    f x  b  LggR x  gf   x    ff   b   ",
    "dddffLgggggggggggg]fffLgggfLgggggggggggggg",
    "ddddddddddddddddddddddddddddddddddddddddd"
  ],
  "winScore": 12,
  "gravity": 10,
  "fallResetMarginTiles": 3,
  "playerStart": { "x": 24, "y": 216 },
  "view": {
    "w": 240, "h": 192,
    "background": [45, 40, 60],
    "cameraLerp": 0.1,
    "parallax": [
      { "img": "assets/background_layer_1.png", "speed": 0.2 },
      { "img": "assets/background_layer_2.png", "speed": 0.4 },
      { "img": "assets/background_layer_3.png", "speed": 0.6 }
    ]
  },
  "overrides": {
    "hud": { "rescueLabel": "RESCUED", "winScoreLabel": "RESCUED %d/%d" }
  }
}
```

7. main.js — Level transition function:

```
async function transitionToLevel(levelId) {
  noLoop();
```

```
    // Remove all sprites safely
    for (const s of [...allSprites]) { s.remove(); }
    // Load new level package
    levelPkg = await cachedLoader.load(LEVELS_URL, levelId);
    assets = await loadAssets(levelPkg, tuningDoc);
    // Rebuild parallax, HUD, Game, UI overlays, particles, camera
    // Re-subscribe all events
    // Re-apply debug gravity if it was on
    soundManager?.stopMusic();
    loop();
  }
```

Also wired in `initRuntime()`:

```
game._onLevelAdvance = async (nextLevelId) => {
  await transitionToLevel(nextLevelId);
};
```

8. WinScreen.js — "Press N for next level":

```
draw({ elapsedMs, ..., hasNextLevel = false } = {}) {
  // ... existing win text ...
  if (hasNextLevel) {
    const msg4 = "Press N for next level";
    // draw in green (#00ff7a)
  }
}
```

9. index.html — Title updated to "GBDA302 Week 9"

Prompt 2 (User)

> If README is not fully satisfy and does not match with the current code, please modify it. And once again, is this code fully satisfy of side quest 09 without any bugs?

Response 2 (Claude Code)

Claude Code performed a verification pass:

1. **Found and fixed Bug 1:** `allSprites.removeAll()` may not exist in p5play v3 — replaced with `for (const s of [...allSprites]) { s.remove(); }`.
2. **Found and fixed Bug 2:** `game._currentLevelIndex = prevLevelIndex` in `transitionToLevel()` was overriding the correct index that `build()` already computes from the level ID. Removed the line.
3. **Verified level 2 tilemap:** Ran a Python script to count entities:

    - Level 1: 15 leaves (winScore=15), 7 boars, 17 fires
    - Level 2: 17 leaves (winScore=12), 10 boars, 17 fires Both levels are completable.

4. **Fixed README:** Changed "Collect all leaves to win" to "Collect enough leaves to win (15 in Level 1, 12 in Level 2)".

Prompt 3 (User)

> please add the full transcript in the genai documentation as a appendix

Response 3 (Claude Code)

Added the full transcript as the Appendix section of the Process & Decision Documentation.

Prompt 4 (User)

> After I pressed N, the another level is not working properly. I mean the fox can not jump at all and only left and right keys are working. Please fix this error and include this part in the README and process and document part please.

Response 4 (Claude Code)

Diagnosed the root cause: `transitionToLevel()` did not re-apply critical p5play world settings after clearing and rebuilding all sprites. Specifically, `world.autoStep = false` was missing, causing the physics engine to double-step each frame (p5play auto-step + our manual `world.step()` in Level.update()). This made the ground-detection sensor desync from the player body, so `isGrounded()` always returned false and jumps were blocked.

Fix applied to main.js — added these lines before `game.build()` in `transitionToLevel()`:

```
pixelDensity(1);
noSmooth();
drawingContext.imageSmoothingEnabled = false;
frameRate(60);
allSprites.pixelPerfect = true;
world.autoStep = false;
```

Updated README.md Post-Playtest notes (item 4) and Process_and_Decision_Documentation.md Limitations or Misfires section (item 3) to document this bug and its fix.

The following is the full transcript of the conversation with the GPT Model used for debugging follow-up, code review, and documentation updates.

Prompt 5 (User)

> I'm currently working on creating a new level for this game. The status right now of the game is that transitioning into the next level after I complete the first level and press N works fine, but in the second level, the motion of the fox (the player character in the game) is weird, which means it does not move in motion as it suppose to and does not jump when I press the appropriate jump key. what is the problem?
>
> The related files for the new level are mostly data/levels.json, src/entities/PlayerEntity.js, src/Level.js, src/LevelLoader.js, and main.js.

Response 5 (GPT Model)

GPT Model inspected the relevant files first instead of assuming the cause. It traced how level 2 was loaded and how player movement and jumping depended on grounded detection, then reported that the likely first problem was not `levels.json` alone but the fox's grounded sensor in `src/entities/PlayerEntity.js`. It explained that `jump()` only runs when `isGrounded()` is true, and that the current player sensor setup looked more fragile than the comparable boar probe setup elsewhere in the project.

Prompt 6 (User)

> In level 1 of the game, the game and everything worked perfectly fine. It's the next level. so what files are you saying that I have to fix it

Response 6 (GPT Model)

GPT Model explained that the first file to try was `src/entities/PlayerEntity.js`, because that is where the fox's ground sensor is created and where `isGrounded()` reads overlaps. It also noted that `data/levels.json` could still matter secondarily if the level 2 spawn was exposing the bug more strongly, but that the player entity was the main suspect.

Prompt 7 (User)

> yes can you please try to fix it? Please ask me before you make changes

Response 7 (GPT Model)

GPT Model agreed to ask for permission before editing and proposed the smallest first change: modify only the fox's ground sensor in `src/entities/PlayerEntity.js`, then test whether level 2 jumping and animation returned without touching other systems.

Prompt 8 (User)

> yes please

Response 8 (GPT Model)

After approval, GPT Model made a first-pass edit to `src/entities/PlayerEntity.js` that changed the fox sensor into a real sensor collider instead of removing its colliders, added a small reset sync, and ran a

syntax check on that file. It then asked for a retest in level 2.

Prompt 9 (User)

> The game looks like this right now. the fox does not jump and doesn't move in motion. Also the green element is floating off the ground

Attached: screenshot of level 2 showing the fox stuck in idle and a floating leaf

Response 9 (GPT Model)

GPT Model looked at the screenshot and separated the two observations. It said the floating green element appeared to be a leaf placement issue in the tilemap rather than a new rendering bug, while the fox still not jumping or switching into run animation continued to point to grounded detection. It proposed that the next likely fix would be a more thorough sensor rewrite that removed the `GlueJoint` approach and manually synced the fox sensor instead.

Prompt 10 (User)

> Okay, I fixed the floating green issue, but if you rewrite the fox sensor, will that be affecting the game in level 1?

Response 10 (GPT Model)

GPT Model explained that changing `src/entities/PlayerEntity.js` would technically affect both levels, because both share the same player code. However, it also clarified that a minimal sensor-only change would aim to keep speed, jump strength, animations, and level data untouched so level 1 would ideally feel the same while only making grounded detection more reliable.

Prompt 11 (User)

> I don't want to change anything for level 1 since it works fine. Can we just use the structure of the initRunctime, which the level 1 of the game gets created? Because for level 2, it would be the same machanics with level one. the only difference is that the location of the leaf items and map itself looks different

Response 11 (GPT Model)

Because the user wanted to avoid changing shared player behavior, GPT Model agreed to try a `main.js`-only approach first. It reviewed the existing `initRuntime()` and transition code and suggested rebuilding level 2 through the same runtime creation path as level 1 before making further edits to `PlayerEntity.js`.

Prompt 12 (User)

> yes let's try that

Response 12 (GPT Model)

GPT Model reverted the earlier `PlayerEntity.js` sensor experiment so level 1 would stay on the original fox code. It then refactored `main.js` so level transitions would rebuild through `initRuntime()` with fresh runtime systems such as `InputManager` and `DebugOverlay`, while preserving transferred debug state. It ran syntax checks on `main.js` and `src/entities/PlayerEntity.js` after the refactor.

Prompt 13 (User)

> it still does not work

Response 13 (GPT Model)

GPT Model treated this failed retest as an important debugging result. It concluded that the level-transition/runtime path was not the remaining root cause, because rebuilding level 2 through the same setup path as level 1 still did not restore jump or run animation. It reported that the likely remaining problem was still the fox's grounded-detection logic in `src/entities/PlayerEntity.js`.

Prompt 14 (User)

> but why is this issue happening only for level 2? Isn't playerentity also used for level 1?

Response 14 (GPT Model)

GPT Model explained that a shared bug can still appear only in one level if the two levels place the player into different conditions. It noted that a fragile grounded sensor might be tolerated by level 1's simpler starting geometry or boot conditions, while level 2's different spawn context or surrounding tiles could expose that weakness immediately.

Prompt 15 (User)

> then if the nearby platform of the starting point is the same, the fox will function correctly? is that what you mean?

Response 15 (GPT Model)

GPT Model said that matching the level 2 start area to level 1 would be a useful test, because jump and run animation both depend on grounded detection. If the fox started functioning correctly on level 2 after that change, it would suggest the map geometry was exposing the issue. If not, the bug would more likely be inside the player grounding logic itself.

Prompt 16 (User)

> I've pasted level 1 and level 2 images. I've corrected the tilemap in levels.json with the same starting point and the tiles. But the fox doesn't move in motion like foxSpriteSheet.png of the fox's motion and it doesn't jump either.

Attached: side-by-side level 1 and level 2 screenshots with matched starting geometry

Response 16 (GPT Model)

After comparing the screenshots and the updated `levels.json`, GPT Model concluded that the level 2 tilemap/start geometry was no longer the main cause. Because level 2 still failed even after its starting area was aligned with level 1, the debugging narrowed further: both missing jump input and missing run animation still pointed to `isGrounded()` and the fox sensor / `GlueJoint` setup inside `src/entities/PlayerEntity.js`.

Prompt 17 (User)

> Can you add these full transcript in the process document (Appendix part) with the same format of the existing file format. And also please replace the gpt contents in Process doc and Readme into this conversation contents.

Response 17 (GPT Model)

GPT Model updated `Process_and_Decision_Documentation.md` to replace the outdated GPT summary with this newer debugging conversation, keeping the same appendix `Prompt / Response` format already used in the document. It also revised the README and the Process document's summary sections so they no longer claim the level 2 control bug is fully fixed, and instead reflect the latest debugging conclusion: reusing the level 1 runtime path and matching the level 2 start area were both useful tests, but the remaining likely cause is still the fox grounded-detection logic in `src/entities/PlayerEntity.js`.

The following is the full transcript of a second Claude Code (Claude Opus 4.6) debugging session, focused on further investigating the level 2 control bug.

Prompt 18 (User)

> I'm trying to build the level 2. Transitioning to level 2 after level 1 is done works, but the character, the fox, does not move in motion with foxSpriteSheet.png and it does not jump. what would be the problem?
>
> Since the level 1 works perfectly fine, I think the transitionToLevel() function can be structured with initRuntime(). What do you think about this approach?

Response 18 (Claude Code)

Claude Code explored the full codebase (main.js, Game.js, Level.js, LevelLoader.js, PlayerEntity.js, PlayerController.js, InputManager.js, WinScreen.js, DebugOverlay.js, AssetLoader.js, TileBuilder.js, levels.json) to understand the level initialization and transition flows.

After comparing `initRuntime()` and `transitionToLevel()`, Claude Code agreed with the user's approach. The old `transitionToLevel()` duplicated approximately 60 lines of setup logic from `initRuntime()` manually, creating a risk of subtle differences in how p5play re-initialized sprites, physics,

and event wiring. By funneling both paths through the same `initRuntime()`, level 2 setup would be guaranteed to match level 1.

Changes applied to main.js:

1. **Simplified `transitionToLevel()`** to only handle the async parts (loading level package, assets, parallax layers, stopping music), then delegate to `initRuntime()` for the actual setup:

```
async function transitionToLevel(levelId) {
  console.log(`TRANSITION: loading level "${levelId}"`);
  noLoop();

  for (const s of [...allSprites]) {
    s.remove();
  }

  if (!cachedLoader) {
    cachedLoader = new LevelLoader(tuningDoc);
  }
  levelPkg = await cachedLoader.load(LEVELS_URL, levelId);
  assets = await loadAssets(levelPkg, tuningDoc);

  const defs = levelPkg.level?.view?.parallax ?? [];
  parallaxLayers = defs
    .map((d) => ({
      img: loadImage(d.img),
      factor: Number(d.speed ?? 0),
    }))
    .filter((l) => l.img);

  soundManager?.stopMusic();
  initRuntime();

  console.log(`TRANSITION: level "${levelId}" ready`);
}
```

2. **Modified `initRuntime()`** to conditionally create systems with `if (!inputManager)` / `if (!debugOverlay)` guards so they are created on first boot but reused across level transitions. Also added debug gravity restoration after `game.build()`:

```
// Systems — only create on first boot; reuse across level transitions
if (!inputManager) inputManager = new InputManager();
if (!debugOverlay) debugOverlay = new DebugOverlay();

// ... game build ...

// Re-apply debug gravity if it was toggled on before this level loaded
if (debugOverlay?.moonGravity) {
  game._applyGravity();
}
```

Prompt 19 (User)

> the jump is not still working

Response 19 (Claude Code)

Claude Code investigated deeper by reading `src/world/TileBuilder.js` and tracing how ground tiles are created and matched to groups. It identified a new root cause: **stale p5play Group objects**.

When `transitionToLevel()` removes all sprites via `s.remove()`, it removes individual sprites from `allSprites` and their physics bodies, but it does NOT remove the `Group` objects themselves. The old groups from level 1 (e.g., `level.ground` with `tile = "g"`, `level.platformsL` with `tile = "L"`) persist in p5play's internal group registry with their `.tile` property still set.

When `buildTilesAndGroups()` runs for level 2, it creates new groups with the same `.tile` characters. But when `new Tiles(tilemap, 0, 0, tileW, tileH)` executes, p5play matches tile characters against all registered groups — including the old empty ones from level 1. If p5play matches a character like `"g"` to the old level 1 ground group instead of the new level 2 ground group, the spawned tile sprites end up in the wrong group.

The resulting symptoms match the bug exactly:

- Level 2 tiles are **visible** (they exist in `allSprites` and are drawn by `allSprites.draw()`)
- But `level.ground`, `level.platformsL`, etc. (the **new** groups referenced by Level) are **empty**
- `isGrounded()` checks `sensor.overlapping(level.ground)` against the new empty groups and always returns `false`
- No jump (requires `isGrounded() === true`), no run animation (requires grounded), only the airborne/fall animation frame plays
- Left/right movement still works (velocity-based, does not require grounded state)

Proposed fix: Remove old level groups from p5play's internal registry before building the new level, so `Tiles()` only finds the newly created groups when matching tile characters. This can be done by calling `group.remove()` on each of the old Level's groups (ground, groundDeep, platformsL, platformsR, wallsL, wallsR, leaf, fire, boar) during the `transitionToLevel()` cleanup phase, before `initRuntime()` creates new ones.

Prompt 20 (User)

> can you put this conversation into the process and document in appendix please as a full transcript? As well please add contents of this in Readme and process doc with the same format with existing file's format

Response 20 (Claude Code)

Claude Code added the full transcript of this debugging session (Prompts 18–20) to the Appendix and updated the Process & Decision Documentation body sections and README to reflect the new stale-groups diagnosis.